

LAB1: 조합논리회로 실습

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/09/14

대상: AND, OR, XOR 게이트/전가산기/4비트 가산기/4비트 감산기/4비트 비교기
/MUX/DEMUX/인코더/디코더/세그먼트 디코더

검증 환경: VS Code VaporView

보고서 및 실험의 범위

기본 논리 게이트부터 산술 연산기, 멀티플렉서, 디코더 등 총 10종의 조합논리회로(실험 01~10)를 대상으로 Verilog HDL 설계와 사전 시뮬레이션을 정리했다. 실제 보드 구현·계측은 수업에서 수행할 항목으로 구분했다.

소스 식별

프로젝트: lab1

기준 커밋: 818e934b77da883164b22f5207b5192c259f3e65

실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab1.git

제출 태그: v1.0

1. 목적과 공통 검증 조건

실험 목적

기본 논리 게이트부터 산술 연산기, 멀티플렉서, 디코더 등 총 10종의 조합논리회로를 Verilog HDL로 설계하고, 시뮬레이션을 통해 진리표와 일치하는지 사전 검증한다.

FPGA와 ASIC의 차이점

FPGA: 록업 테이블, 플립플롭, 프로그래머블 인터커넥트 스위치 블록으로 구성된 사전 제작된 범용 실리콘 다이이다. 설계 수정이 자유롭다는 장점이 있다.

ASIC: 특정 애플리케이션에 맞추어 실리콘 웨이퍼 상에 트랜지스터와 금속 배선을 영구적으로 제작하는 방식으로, 최적의 면적, 최고 수준의 동작 속도, 최소 전력 소모를 달성할 수 있으나, 한 번 제조된 이후에는 회로의 논리 구조를 물리적으로 수정할 수 없다.

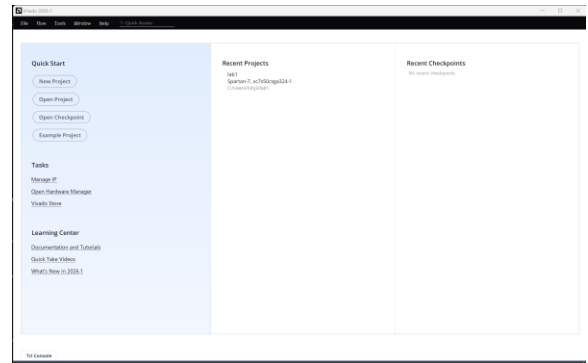
반도체 공정 관점의 FPGA 사용 이유

대표적 정량 연구인 Kuon과 Rose에 따르면, FPGA는 프로그래밍 오버헤드로 인해 동일 공정 표준 셀 ASIC 대비 약 20~35배 넓은 실리콘 면적을 차지하고 동작 지연 시간도 3~4배 길다 [1].

이러한 하드웨어적 불리함에도 불구하고 초기 검증 및 실습 단계에서 FPGA를 사용하는 핵심 이유는 반도체 공정 경제성에 있다. 첨단 공정일수록 마스크 제작을 비롯한 초기 비반복 엔지니어링(NRE) 비용이 수백억 원대에 달하며, 설계 오류로 인한 재제작(Re-spin) 시 막대한 공정 지연과 손실이 발생한다 [1]. 반면 FPGA는 이미 양산된 범용 칩을 재프로그래밍하므로 마스크 제작 비용(Zero NRE)이 발생하지 않으며, 물리적 팹(Fab) 제작 기간을 거치지 않

아 프로토타이핑 검증 및 출시 기간(Time-to-Market)을 획기적으로 단축시킬 수 있다 [1].

Vivado 2026.1 설치 확인



2. 실험1: AND, OR, XOR 논리 게이트 동작 원리

out_and=A·B: 두 입력이 모두 1일때만 1을 출력

out_or=A+B: 입력 중 하나라도 1이면 1을 출력

out_xor=A⊕B: 두 입력의 값이 서로 다를때만 1을 출력

핵심 코드

```
assign out_and = a & b;
```

```
assign out_or = a | b;
```

```
assign out_xor = a ^ b;
```

소스 코드 구현 설명

src/logic_gates.v에서는 1비트 입력선 a,b를 선언하고 연속 할당문 assign과 비트 연산자 &,|,^를 이용해 조합논리를 단일 수준으로 직관적으로 모델링하였다.

sim/tb_logic_gates.sv에서는 initial 블록 내부에서 2비트 변수를 루프를 통해 10ns 간격으로 00->01->10->11 순서로 전수 인가하도록 구성하였으며, \$dumpfile ("wave.vcd")를 호출해 VaporView에서 파형을 확인할 수 있도록 설정하였다.

예상 결과

0~10ns(A=0,B=0): out_and=0, out_or=0,

out_xor=0

10~20ns(A=0,B=1): out_and=0, out_or=1,

out_xor=1

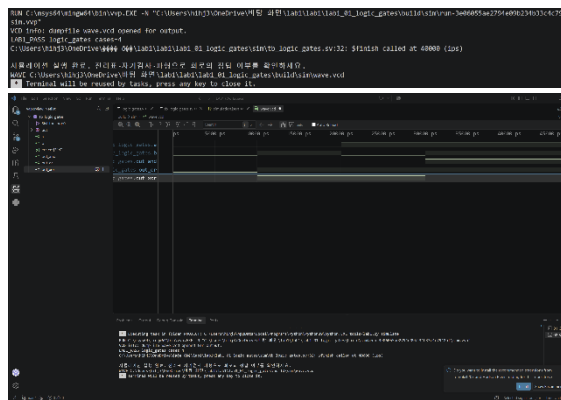
20~30ns(A=1,B=0): out_and=0, out_or=1,

out_xor=1

30~40ns(A=1,B=1): out_and=1, out_or=1,

out_xor=0

시뮬레이션 결과 분석



파형 상세 분석

0~10ns (A=0, B=0): 모든 게이트의 출력이 0을 유지함을 확인.

10~20ns (A=0, B=1) 및 20~30ns(A=1, B=0): out_or와 out_xor가 1로 천이하고 out_and는 0을 유지함을 확인.

30~40ns (A=1, B=1): out_and가 1로 올라가고, 두 입력이 동일하므로 out_xor는 0으로 반전되어 진리표와 완전히 일치함을 확인.

3. 실험2: 전가산기

동작 원리

1차 반가산기: $S_1 = A \oplus B$, $C_1 = A \cdot B$

2차 반가산기: $S = S_1 \oplus C_{in}$, $C_2 = S_1 \cdot C_{in}$

최종 캐리: $C_{out} = C_1 + C_2$

핵심 코드

```
half_adder u_ha1 (.a(a), .b(b), .s(s1), .c(c1));
```

```
half_adder u_ha2 (.a(s1), .b(cin), .s(s), .c(c2));
```

```
assign cout = c1 | c2;
```

예상 결과

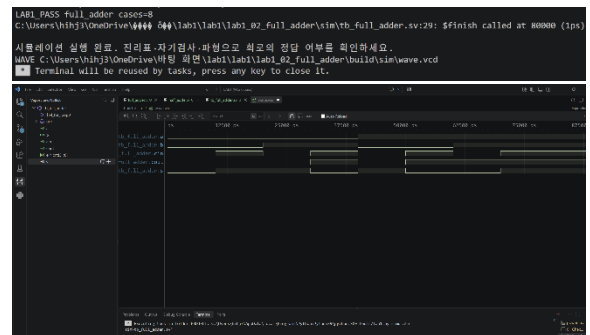
0~10ns(A=0, B=0, Cin=0): S=0, Cout=0

입력 중 1개만 1인 구간: S=1, Cout=0

입력 중 2개만 1인 구간: S=0, Cout=1

입력 3개 모두 1인 구간: S=1, Cout=1

시뮬레이션 결과 분석



파형 상세 분석

30ns (A=0, B=1, $C_{in}=1$): 입력 중 1이 2개 인가되었을 때 합 $S=0$, 올림수 $C_{out}=1$ 이 출력되어 캐리가 정상 발생함.

70ns (A=1, B=1, $C_{in}=1$): 3개 입력이 모두 1일 때 $S=1$ 과 $C_{out}=1$ 이 동시에 출력되어 이진 덧셈 $1+1+1=11(2)$ 연산이 일치함을 입증.

4. 실험3: 4비트 가산기

동작 원리

내부적으로 최하위 비트의 캐리 입력을 0으로 고정한 상태에서 덧셈을 수행하며, 4비트 연산 결과에서 발생하는 오버플로우는 최상위 1비트 올림수 출력(cout)으로 내보내고 나머지 4비트 결과는 합(s[3:0])으로 출력된다.

핵심 코드

```
assign {cout, s} = a + b;
```

소스 코드 구현 설명

외부 cin 입력 포트를 두지 않고, 4비트 입

력 벡터 $a[3:0]$ 와 $b[3:0]$ 의 덧셈 결과를 5비트 결합 연산자 $\{cout, s\}$ 에 직접 할당하였다. 최상위 1비트는 올림수($cout$), 하위 4비트는 합(s)으로 자동 처리되어 별도의 캐리 입력 제어 없이도 안정적인 덧셈 연산이 수행되도록 구현하였다.

예상 결과

초기상태: $s=4'b0000$, $cout=0$

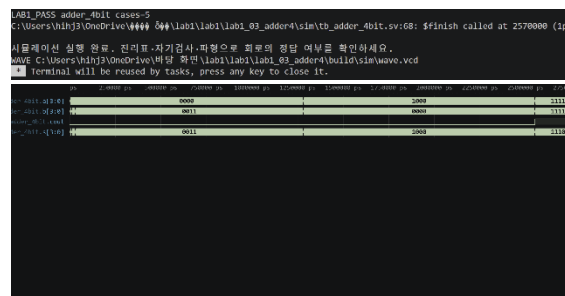
일반가산(예: $A=4'd7$, $B=4'd5$, $Cin=0$):

$s=4'b1100$, $cout=0$

최댓값 가산(예: $A=4'd15$, $B=4'd15$, $Cin=1$):

$s=4'b1111$, $cout=1$

시뮬레이션 결과 분석



5. 실험4: 4비트 감산기

동작 원리

4비트 피감수 A 에서 감수 B 를 빼는 회로로, 2의 보수 감산 원리($A-B=A+\sim B+1$)를 바탕으로 차이값($diff$)과 상위 자리로부터의 빌림수($borrow$)를 연산하도록 구현하였다.

핵심 코드

```
assign {borrow, diff} = {1'b0, a} - {1'b0, b};
```

소스 코드 구현 설명

```
assign {borrow, diff} = {1'b0, a} - {1'b0, b};
```

입력 비트보다 1비트 큰 5비트 폭으로 확장하여 감산 연산을 수행하였다. 하위 4비트는 뺄셈 결과인 $diff$ 에 할당되고, 최상위 비트는 $A < B$ 일 때 음수 표현을 위해 발생

하는 빌림수 $borrow$ 로 분리 추출되도록 데이터플로우 방식으로 모델링하였다.

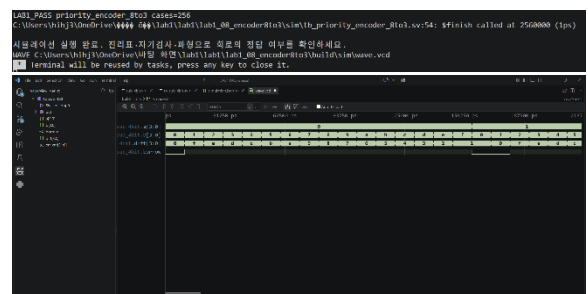
예상 결과

A가 B보다 큰 구간: 빌림수 없이 양수 차이 계산

A와 B가 같은 구간: $diff=4'd$, $borrow=0$

A가 B보다 작은 구간: $borrow=1$

시뮬레이션 결과 분석



파형 상세 분석

A가 B보다 크거나 같은 구간: $borrow=0$ 이 출력되어 빌림수 없이 뺄셈이 수행된다.

A가 B보다 작은 구간: 음수 결과 발생으로 인해 최상위 빌림수 비트 $borrow=1$ 이 활성화되고, 각각 2의 보수 표현에 일치하는 결과가 도출된다.

6. 실험5: 4비트 크기 비교기

동작 원리

두 개의 4비트 부호 없는 이진수 $A[3:0]$ 와 $B[3:0]$ 의 크기를 비교하여 3가지 상태 플래그(lt , eq , gt)를 출력한다.

핵심 코드

```
assign lt = (a < b);
```

```
assign eq = (a == b);
```

```
assign gt = (a > b);
```

소스 코드 구현 설명

관계 연산자 ($<$, $=$, $>$)를 사용하여 A와 B의 크기 관계를 '미만(lt)', '같음(eq)', '초과(gt)'의 상호 배타적인 3개 플래그로 출력

함

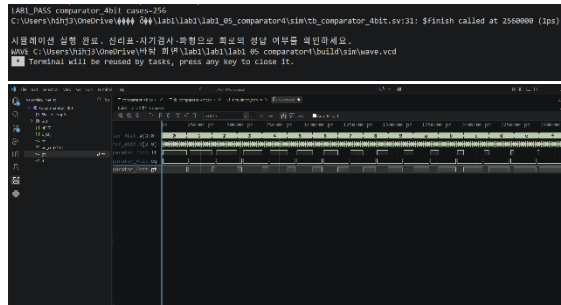
예상 결과

A<B: lt=1, eq=gt=0 출력

A=B: lt=gt=0, eq=1 출력

A> B: lt=eq=0, gt=1 출력

시뮬레이션 결과 분석



파형 상세 분석

A<B: lt=1, eq=gt=0 출력

A=B: lt=gt=0, eq=1 출력

B> B: lt=eq=0, gt=1 출력

➔ 세 플래그 신호가 겹치지 않으며 예상 결과와 같음을 확인.

7. 실험6: 4:1 멀티플렉서

동작 원리

4개의 데이터 입력선 d[3:0] 중 2비트 선택 신호 sel[1:0]가 지정하는 단 하나의 입력을 단일 출력선 y로 연결하는 데이터 선택기 (Data Selector)이다.

핵심 코드

```
always @(*) begin
    case (sel)
        2'b00: y = d[0];
        2'b01: y = d[1];
        2'b10: y = d[2];
        2'b11: y = d[3];
        default: y = 1'b0;
    endcase
end
```

소스 코드 구현 설명

조합논리 표현을 위해 always @(*) 블록 내부에 case 문을 구성하였다.

예상 결과

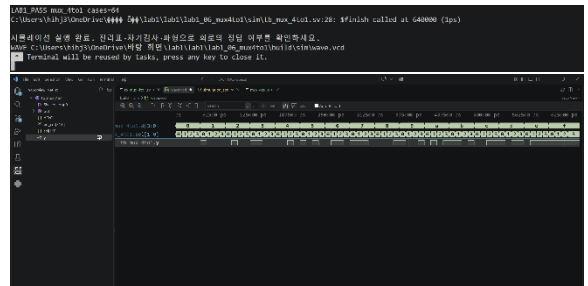
Sel=2'b00: y=d[0]=0

Sel=2'b01: y=d[1]=1

Sel=2'b10: y=d[2]=0

Sel=2'b11: y=d[3]=1

시뮬레이션 결과 분석



8. 실험7: 1:8 디멀티플렉서

동작 원리

멀티플렉서의 역동작 회로로, 단일 데이터 입력선 din을 3비트 선택 신호 sel[2:0]에 따라 8개의 출력선 y[3:0] 중 지정된 하나의 채널로만 전달하는 데이터 분배기이다.

핵심 코드

```
always @(*) begin
    y = 8'b00000000;
    case (sel)
        3'b000: y[0] = din;
        3'b001: y[1] = din;
        3'b010: y[2] = din;
        3'b011: y[3] = din;
        3'b100: y[4] = din;
        3'b101: y[5] = din;
        3'b110: y[6] = din;
        3'b111: y[7] = din;
    endcase
end
```

소스 코드 구현 설명

Always @(*) 블록 시작 부분에서 출력 벡터 전체를 8'b00000000으로 기본 초기화하여 래치 생성을 방지하였다.

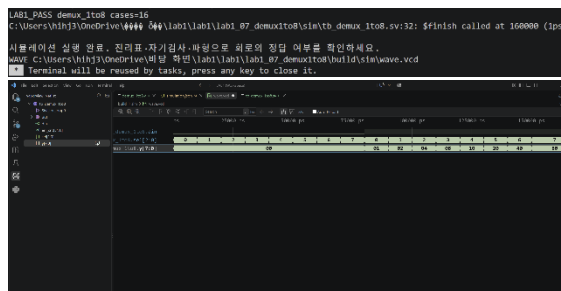
예상 결과

Din=0구간: sel값과 무관하게

y=8'b00000000 유지

Din=1 구간: LSB부터 차례대로 한 비트만
1로 천이

시뮬레이션 결과 분석



파형 상세 분석

din=1일 때 sel을 0부터 7까지 변경:
y가 00000001->00000010->...->10000000
으로 출력.

din=0: sel값과 상관없이 모든 출력이 00000000으로 차단됨.

9. 실험8: 8:3 인코더

동작 원리

8개의 입력선 $i[7:0]$ 중 1로 활성화된 단 하나의 입력 핀에 대응하여 3비트 2진 코드 출력 $a[2:0]$ 을 생성하는 조합논리회로이다. $i[0]$ 단독 활성화(00000001) 시 111을 출력하며, 유효하지 않은 입력(전체 0)이나 두 개 이상의 비트가 동시에 1이 되는 다중 입력 상태에서는 기본값인 000을 출력하도록 설계하였다.

핵심 코드

always $@(*)$ begin
case (i)

```
8'b1000_0000: a = 3'b000;
```

```
8'b0100_0000: a = 3'b001;
```

```
8'b0010_0000: a = 3'b010;
```

```
8'b0001_0000: a = 3'b011;
```

```
8'b0000_1000: a = 3'b100;
```

```
8'b0000_0100: a = 3'b101;
```

```
8'b0000_0010: a = 3'b110;
```

```
8'b0000_0001: a = 3'b111;
```

```
default:      a = 3'b000;
```

endcase

end

소스 코드 구현 설명

case문을 사용하여 i 가 0이 아닐 때에만 a 가 값을 갖도록 하였다.

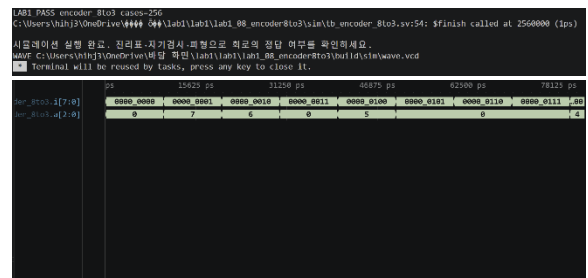
예상 결과

무효입력상태: $i=8b'00000000$, $a=3'b000$

단일입력상태: $i=8'b000000001$, $a=3'b111$

다중입력상태: $i=8'b11111111$, $a=3'b000$

시뮬레이션 결과 분석



파형 상세 분석(예시)

$$l=8'b00000000: a=3'b000$$

$l=8'b00000001$: $a=3'b111$

$$l=8'b00000011: a=3'b000$$
$$l=8'b10000000: a=3'b000$$

10. 실험9: 3:8 디코더

동작 원리

8-to-3 인코더의 역방향 연산을 수행하는 회로로, 3비트 2진 코드 입력 in[2:0] (000(2)~111(2))을 해독하여 총 $2^3=8$ 개의

출력선 out[7:0] 중 해당 번호의 출력 핀 1개만 1로 활성화시킨다.

핵심 코드

```
always @(*) begin
    if (!len) begin
        out = 8'b00000000;
    end else begin
        case (in)
            3'b000: out = 8'b00000001;
            3'b001: out = 8'b00000010;
            3'b010: out = 8'b00000100;
            3'b011: out = 8'b00001000;
            3'b100: out = 8'b00010000;
            3'b101: out = 8'b00100000;
            3'b110: out = 8'b01000000;
            3'b111: out = 8'b10000000;
        endcase
    end
end
```

소스 코드 구현 설명

If(!len) 문을 최상위 조건문으로 배치하여 비활성화 상태를 먼저 방어하고, 활성화 시 case 문으로 매핑을 수행하였다.

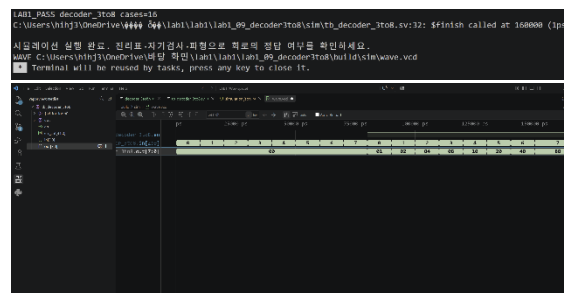
예상 결과

En=0: 입력과 관계 없이

out[7:0]=8'b00000000

En=1: 입력 2진수 크기에 해당하는 단 1개 비트만이 활성화

시뮬레이션 결과 분석



파형 상세 분석

En=0: 입력 in이 바뀌어도 출력 out은 8'b00000000 유지

En=1: 입력 in이 0부터 7로 순차 증가함에 따라 출력 out의 0번 핀부터 7번 핀까지 단 1개의 비트만 순차적으로 1로 전환됨을 확인.

11. 실험10: 7세그먼트 디코더

동작 원리

4비트 BCD(입력 범위 0~15, 16진수 0~F) 입력을 받아 8개의 LED 세그먼트 막대(a, b, c, d, e, f, g, dp)로 구성된 7-세그먼트 표시 장치에 숫자 및 영문자가 형상화되도록 점등 패턴을 변환하는 조합논리회로이다. Active-High 구동 방식을 채택하여 신호가 1일 때 해당 LED 세그먼트가 점등된다.

핵심 코드

```
// seg_data = {a, b, c, d, e, f, g, dp}
```

(Active-High: 1일 때 점등)

```
always @(*) begin
    case (bcd)
        4'h0: seg_data = 8'b1111_1100;
        4'h1: seg_data = 8'b0110_0000;
        4'h2: seg_data = 8'b1101_1010;
        4'h3: seg_data = 8'b1111_0010;
        4'h4: seg_data = 8'b0110_0110;
        4'h5: seg_data = 8'b1011_0110;
        4'h6: seg_data = 8'b1011_1110;
        4'h7: seg_data = 8'b1110_0000;
        4'h8: seg_data = 8'b1111_1110;
        4'h9: seg_data = 8'b1111_0110;
        4'hA: seg_data = 8'b1110_1110;
        4'hB: seg_data = 8'b0011_1110;
        4'hC: seg_data = 8'b1001_1100;
        4'hD: seg_data = 8'b0111_1010;
        4'hE: seg_data = 8'b1001_1110;
```

```

4'hF: seg_data = 8'b1000_1110;
default: seg_data = 8'b0000_0000;
endcase
end

```

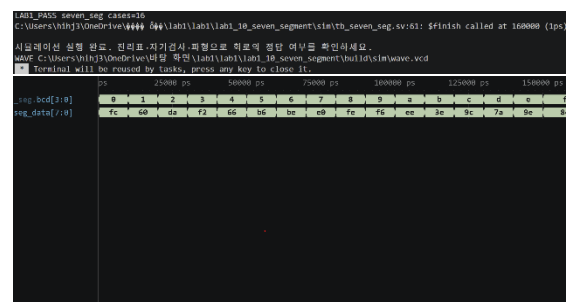
소스 코드 구현 설명

비트 배열 순서를 MSB부터 {a, b, c, d, e, f, g, dp}로 배치하였으며, 단순 0~9에 그치지 않고 10~15에 대응하는 16진수 문자(A~F)까지 16개 케이스 전체를 case 문으로 일대일 매핑하였다. 소수점(dp)은 표시하지 않으므로 항상 0으로 유지된다.

예상 결과

숫자 0: 막대 g 제외 점등
 숫자 1: 막대 b,c만 점등
 숫자 3: 막대 a,b,c,d,g 점등
 숫자 8: 7개 막대 전체 점등

시뮬레이션 결과 분석



른 출력값의 변화를 확인하고, 이론적 진리표와 일치하는 대조하여 시뮬레이션 통과 확인

13.참고 문헌

[1] I. Kuon and J. Rose, "Measuring the Gap Between FPGAs and ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 203-215, Feb. 2007.

12.실험 수행 계획

1단계(프로젝트 생성 및 코딩)

Vivado를 실행하여 실습 대상 회로의 RTL 프로젝트를 생성하고 Design Sources에 Verilog 모듈 코드 작성

2단계(테스트벤치 구동 및 기능 검증)

Simulation Sources에 테스트벤치를 추가하고 Run Behavioral Simulation을 실행하여 Vivado 내장 시뮬레이터로 출력파형 확인

3단계(결과 파형 분석 및 검증)

Vivado Waveform 창에서 입력 벡터에 따